Multiprocessor System

Prepared By Er Abhishek Singh
Computer Engineer
Department of IT
Government of Nepal

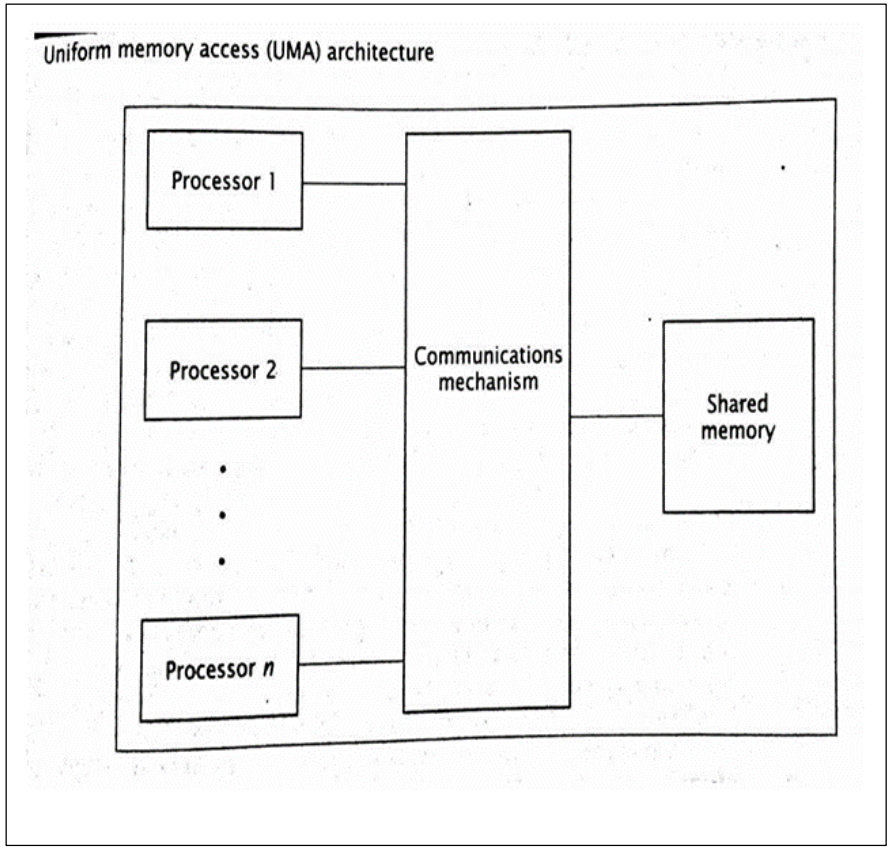
Chapter #8 Multiprocessors

- **Multiprocessor System Characteristics:**
- A multiprocessor system is an interconnection of two or more CPUs with memory and input-output equipment.
- The term “processor” in multiprocessor can mean either a central processing unit (CPU) or an input-output processor (IOP).
- Multiprocessors are classified as multiple instruction stream, multiple data stream (MIMD) systems.
- A multiprocessor system is **controlled by one operating system** that provides interaction between processors and all the components of the system cooperate in the solution of a problem.
- Multiprocessing improves the reliability of the system.
- Multiprocessing can improve performance by decomposing a program into parallel executable tasks.

MIMD

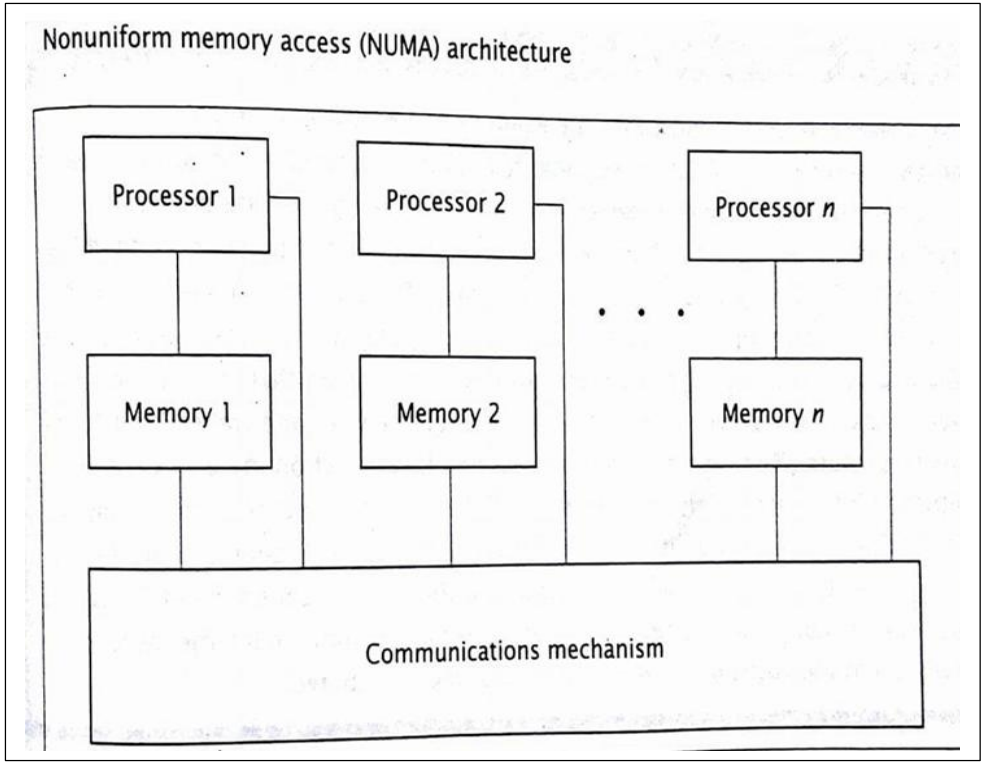
- Refers to the connection of the processors w.r.t System Memory
- Each processor may have its own cache memory, system memory. not directly accessible by the other processors.
- A Symmetric multiprocessor, or SMP, is a computer system that has two or more processors with comparable capabilities (do not have to be identical).
- All processors must be capable of performing the same functions; this is the symmetry of SMPs.
- The processors all have access to the same I/O devices and memory modules.
- One type of SMP is the Uniform Memory Access (UMA) architecture.
- UMA gives all CPUs equal (uniform) access to all locations in shared memory.
- They interact with shared memory via some communication mechanism.

Each processor may have its own cache memory, not directly accessible by the other processors



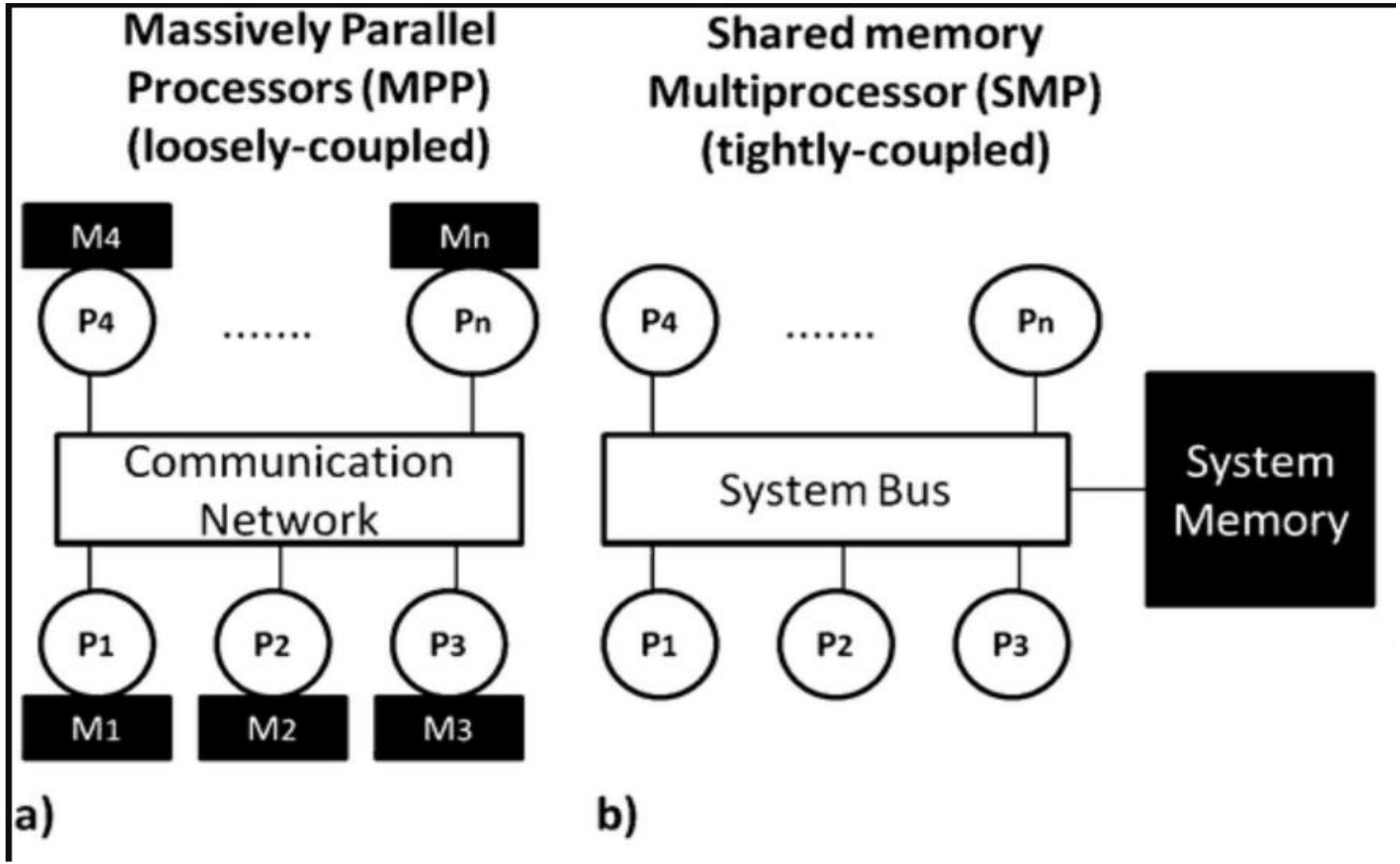
Non-uniform Memory Access (NUMA) do not allow uniform access to all shared memory locations.

- The architecture still allows all processors to access all shared memory locations.
- However, each processor can access the memory module closest to it, its local shared memory, more quickly than the other modules; hence, the memory access times are non-uniform.
- Unlike UMA machines, NUMA computers may not be SMP



Types of Multiprocessors System

- Multiprocessor are classified by the way their memory is organized.
- A multiprocessor system with common shared memory is classified as a shared-memory or tightly coupled multiprocessor.
  - Efficient when interaction between tasks is maximum
- Each processor element with its own private local memory is classified as a distributed-memory or loosely coupled system.
  - Are most efficient when the interaction between tasks is minimal


Massively Parallel Processors (MPP) (loosely-coupled)
Shared memory Multiprocessor (SMP) (tightly-coupled)
M4
Mn
P4
.....
Pn
Communication Network
P1
P2
P3
M1
M2
M3
a)
P4
.....
Pn
System Bus
System Memory
P1
P2
P3
b)

Differences between Loosely coupled and tightly coupled

S. No.	Loosely Coupled Multiprocessor System	Tightly Coupled Multiprocessor System
1.	In this system, every processor has its own memory module.	In this system, the processors share memory modules.
2.	It is efficient when there is less interaction between tasks running on different processors.	It is efficient when used with real time processing.
3.	There are no memory conflicts in general.	It has memory conflicts.
4.	It is considered as a Message transfer system (MTS).	They are connected through networks such as PMIN, IOPIN, and ISIN.
5.	It is less expensive.	It is expensive.
6.	It has a low data rate.	It has a high data rate.
7.	It provides comparatively slow speed.	It provides high speed
8.	They are usually seen in distributed computing systems	It is usually seen in parallel processing systems.
9	In loosely coupled multiprocessor, Memory conflicts don't take place	While tightly coupled multiprocessor system have memory conflicts
10	Loosely Coupled Multiprocessor system has low degree of interaction between tasks	Tightly Coupled multiprocessor system has high degree of interaction between tasks

Interconnection Structure

- The components that form a multiprocessor system are CPUs, IOPs connected to input- output devices, and a memory unit.
- There are several physical forms available for establishing an interconnection network.
  - Time-shared common bus
  - Multiport memory
  - Crossbar switch
  - Multistage switching network
  - Hypercube system

Time Shared Common Bus

- A common-bus multiprocessor system consists of a number of processors connected through a common path to a memory unit.
- Only one processor can communicate with the memory or another processor at any given time.

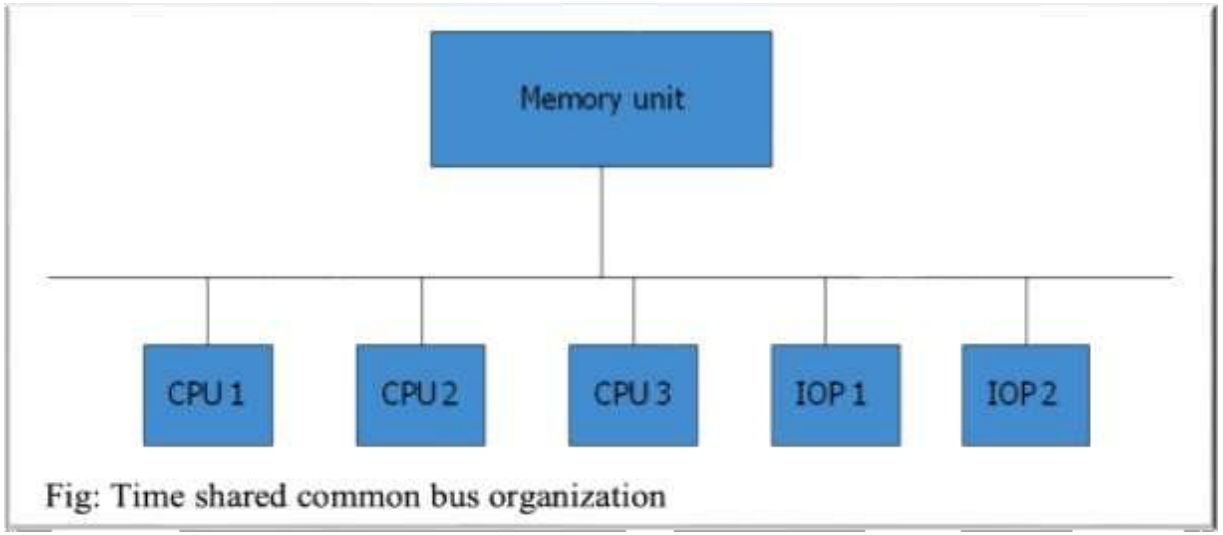


Fig: Time shared common bus organization

Multiport Memory

- A multiport memory system employs separate buses between each memory module and each CPU.
- The module must have internal control logic to determine which port will have access to memory at any given time.

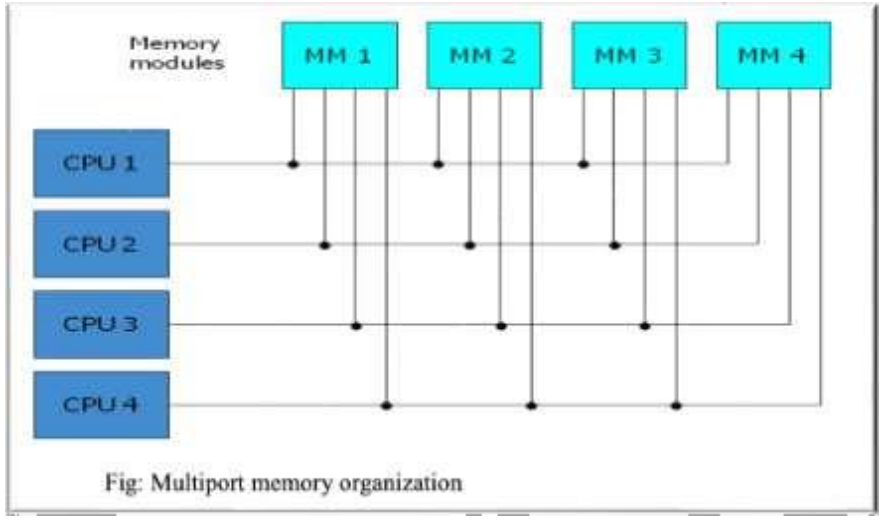


Fig: Multiport memory organization

Crossbar Switch

- Consists of a number of crosspoints that are placed at intersections between processor buses and memory module paths.
- The small square in each crosspoint is a switch that determines the path from a processor to a memory module.

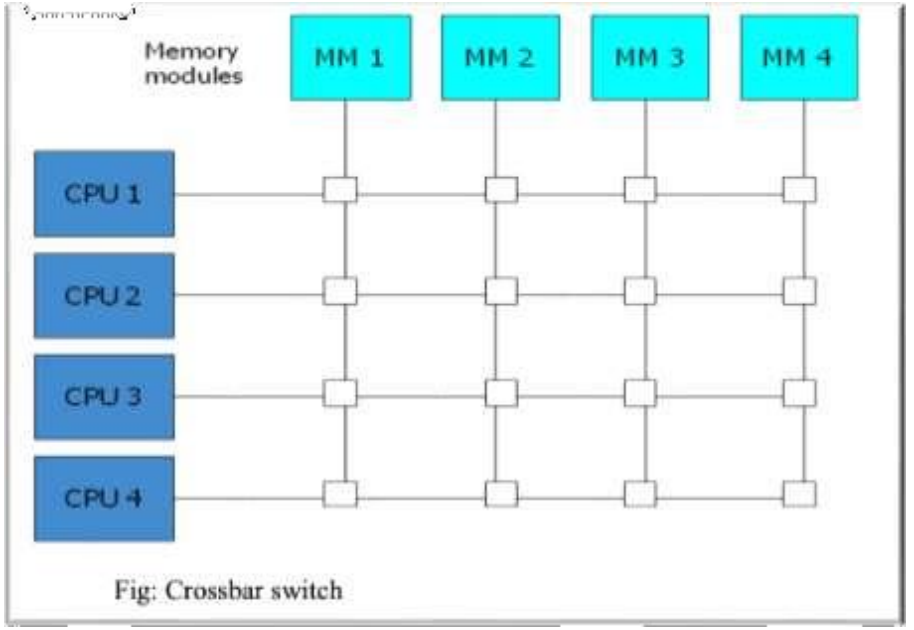
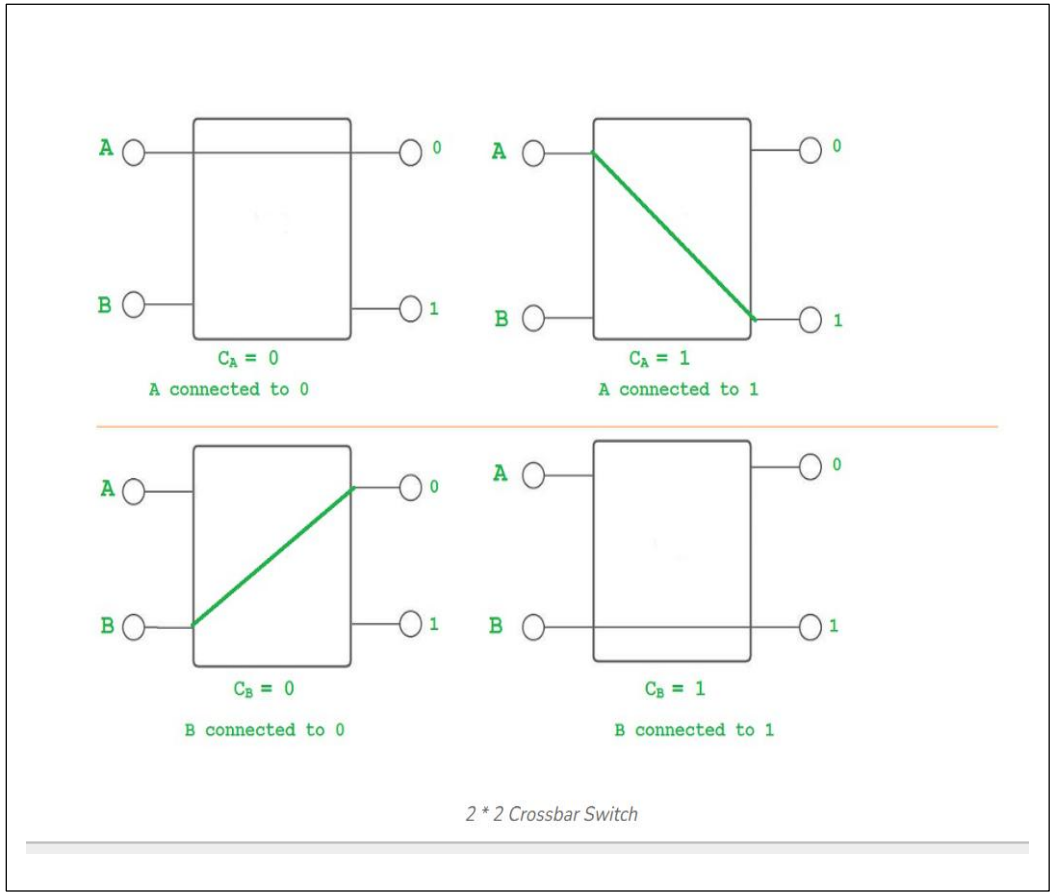
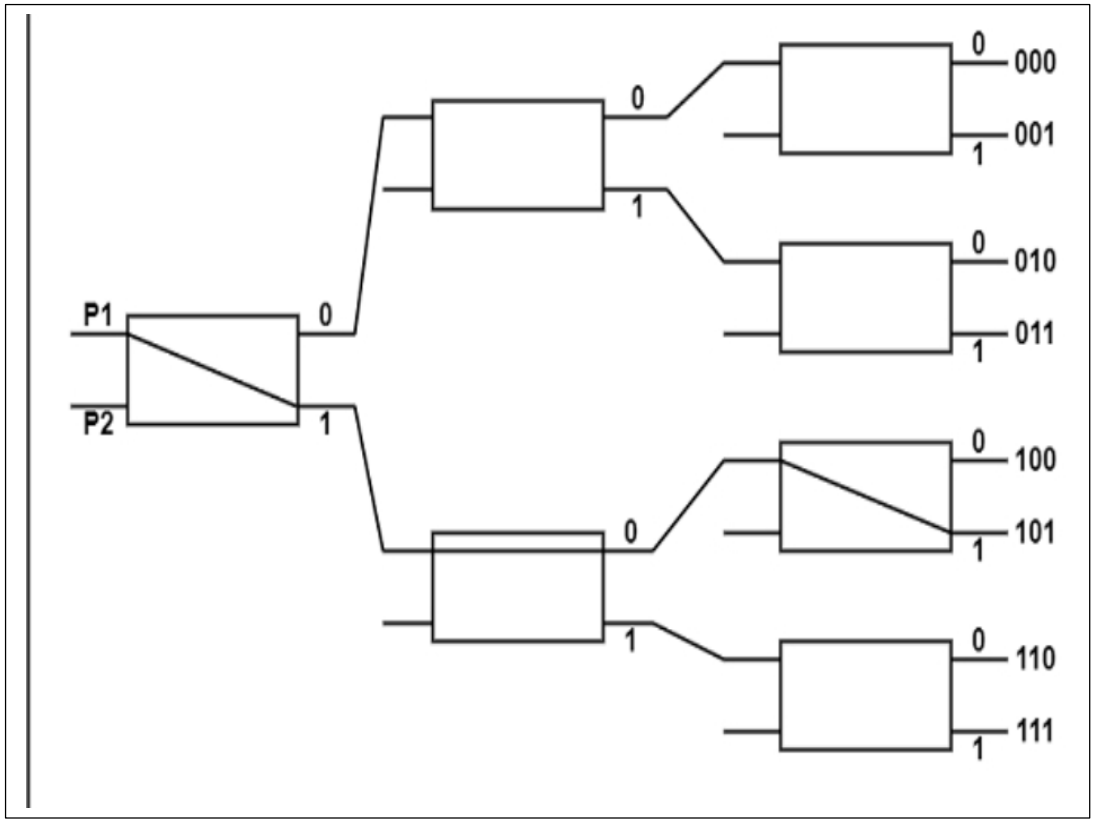


Fig: Crossbar switch

Multistage Switching Network





A Multistage Network is used in multiprocessor systems to connect multiple processors to multiple memory modules efficiently.

Instead of using:

A single shared bus (slow), or

A full crossbar switch (very expensive),

The basic element of a multistage network is a two-input, two-output interchange switch. As displayed in the figure, the 2 X 2 switch has two inputs labeled A and B, and two outputs labeled 0 and 1.

There are control signals related to the switch. The control signals start interconnection between the input and output terminals.

In case, inputs A and B request the same output terminals, it is possible that only one of the inputs is connected and the other is blocked. It is possible to start a multistage network to control the communication between multiple sources and destinations.

Consider the binary tree displayed in the figure to view how this is carried out.

The two processors P1 and P2 are connected through switches to eight memory modules labeled in binary, starting from 000 through 111. The direction beginning from source to destination is decided from the binary bits of the destination number.

The first bit of the destination number helps in denoting the first level's switch output. The second bit identifies the second level's switch output, and the third bit defines the third level's switch output.

Using the 2x2 switch as a buildings block, it is possible to build a multistage network to

Control the communication between a number of sources and destinations.

To see how this is done, consider the binary tree as shown in Fig.

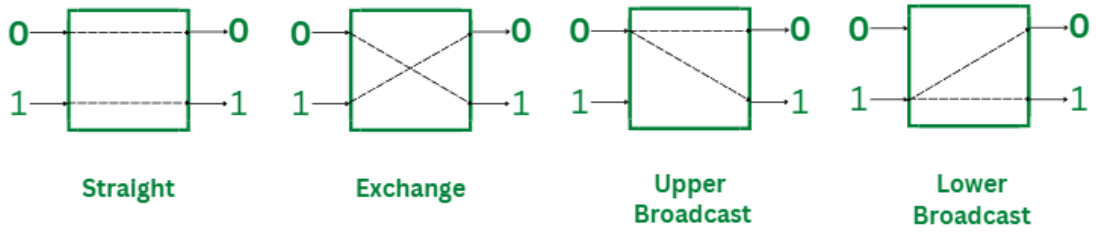
The Omega Switching Network is a type of Multistage Interconnection Network (MIN) used in multiprocessor systems to connect:
Multiple processors
Multiple memory modules
It is cost-effective compared to a crossbar switch.
If we have:
N inputs N outputs
Then the Omega network has:

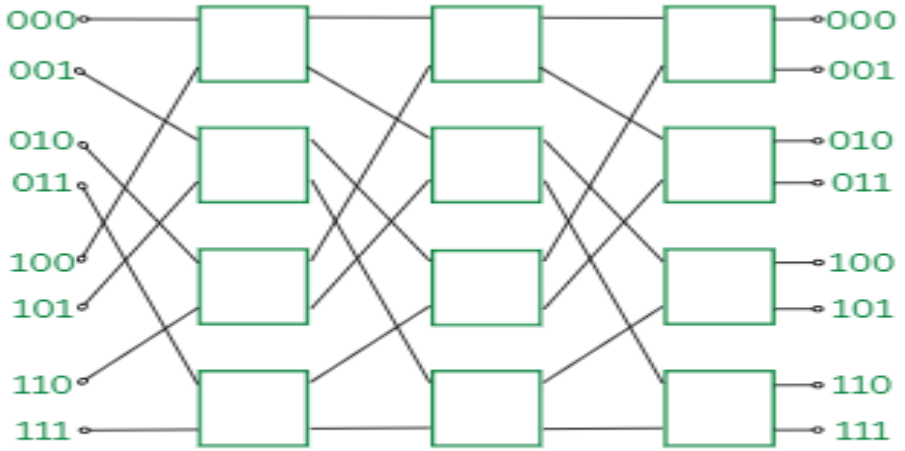
$\log_2(N)$ stages
Each stage contains N/2 switchesFor **N = 8**:

- Number of stages = $\log_2(8)$ = **3**
- Each stage has 4 switches (2×2)
- Each stage consists of N/2 switch boxes. Each switch box is in any of the four states (i.e. straight, exchange, upper broadcast, or lower broadcast).

There is perfect shuffle exchange interconnection between every adjacent stage.
The interconnections between the stages in an Omega Network are defined by the "rotate left" of the bits.

000 -> 000 -> 000 -> 000
001 -> 010 -> 100 -> 001
010 -> 100 -> 001 -> 010
100 -> 001 -> 010 -> 100
011 -> 110 -> 101 -> 011
101 -> 011 -> 110 -> 101
110 -> 101 -> 011 -> 110
111 -> 111 -> 111 -> 111





8 x 8 Omega Network

Hypercube System

- The hypercube or binary n-cube multiprocessor structure is a loosely coupled system composed of $N=2^n$ processors interconnected in an n-dimensional binary cube.
- Each processor forms a node of the cube, in effect it contains not only a CPU but also local memory and I/O interface.
- Fig. below shows the hypercube structure for n=1, 2, and 3.

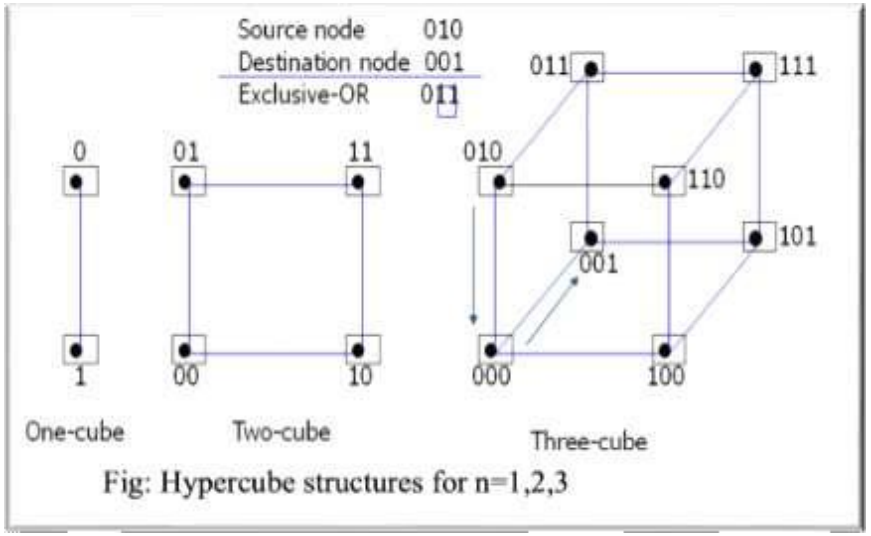


Fig: Hypercube structures for n=1,2,3

A one-cube structure includes n = 1 and 2n = 2. It has two processors that are connected by an individual route. A two-cube structure includes n = 2 and 2n = 4. It has four nodes that are linked as a square. There are eight nodes associated as a cube in a three-cube structure.

There are 2n nodes in an n-cube structure with a processor actual in each node. A binary address is authorized to each node including the addresses of two neighbours vary in particularly one-bit position.
Routing messages through an n-cube structure can need one to n links, beginning from a source node to a destination node. For example, node 000 can interact directly with node 001 in a three-cube structure. It can communicate from node 000 to node 111, the message has to transit through a minimum of three links.

A routing phase can be developed by computing the exclusive-OR of the source node address with the destination node address. The resulting binary value will have 1 bit equivalent to the axes on which the two nodes differ.

For example, in a three-cube structure, a message at 010 going to 001 makes an exclusive-OR of the two addresses similar to 011. The message can be transmitted along the second axis to 000 and then by the third axis to 001.

Each node includes a CPU, a floating-point processor, local memory, and serial communication interface units.

The individual nodes operate independently on data stored in local memory according to resident programs. The data and programs to each node come through a message-passing system from other nodes or a cube manager.

Comparative Study of All these Structure

Time-Shared Bus Multiprocessor
Processors share a single memory via a common bus
Only one processor can access memory at a time
Access is controlled by bus arbitration
Pros:
Simple and cheap
Easy to implement for small systems
Works well for a few processors
Cons:
Only one processor can access memory at a time → bottleneck
Poor scalability (bus becomes saturated)
Slow for large systems

Multistage Network (e.g., Omega Network)
Uses multiple stages of 2×2 switches
Connects multiple processors to multiple memory modules
Pros:
Moderate hardware cost compared to crossbar
Parallel access possible , Structured and scalable
Good performance for moderate-sized systems

Cons:
Blocking can occur (some paths may be busy) Only one path between input-output pair
More complex than simple bus

Multiport Memory
Memory has multiple ports, each processor has a dedicated port
Allows simultaneous access by multiple processors
Pros
Multiple processors can access memory at the same time
Fast memory access
Reduces contention compared to a bus
Cons:
Very expensive for large systems
Complex hardware (multiple address/data lines)
Scalability limited by hardware cost

EErPPr

Hypercube Multiprocessor
Processors connected in an n-dimensional cube
Each processor connected to n others (for 2^n processors)
Pros:
Highly regular structure → easy to scale logically , Logarithmic diameter → fast communication
Good fault tolerance -Efficient parallel communication
Cons:
Hardware complexity increases exponentially with dimension
Wiring and physical layout difficult for large n and routing algorithms can be complex

Inter Processor Arbitration in Multiprocessor

Inter Processor Arbitration

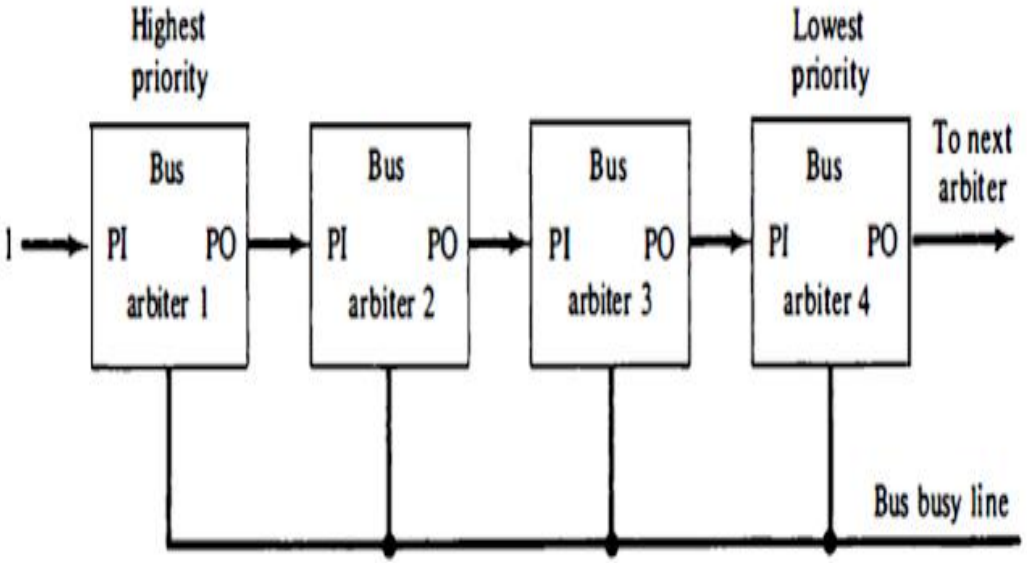
The processor, main memory and I/O devices can be interconnected by means of a common bus. A bus is set of lines (wires) defined to transfer all bits of a word from a specified source to a specified destination. Thus, bus provides a communication path for the transfer of data.

The bus includes data lines, address lines and control lines. Such a bus known as system bus. Different types of arbitration: ***Serial (Daisy Chain) arbitration, Parallel arbitration, Dynamic arbitration.***

Serial (Daisy Chain) arbitration

In this type of arbitration, processors can access bus based on priority. In serial arbitration, bus access priority resolving based on the serial connection of the processors. This technique is obtained from daisy chain (serial) connection of processors. The serial priority resolving technique is obtained from daisy-chain connection similar to the daisy chain priority interrupt logic. The processors connected to the system bus are assigned priority according to their position along the priority control line.

When multiple devices concurrently request the use of the bus, the device with the highest priority is granted access to it. Each processor has its own bus arbiter logic with priority-in and priority-out lines. The priority out (PO) of each arbiter is connected to the priority in (PI) of the next-lower-priority arbiter.

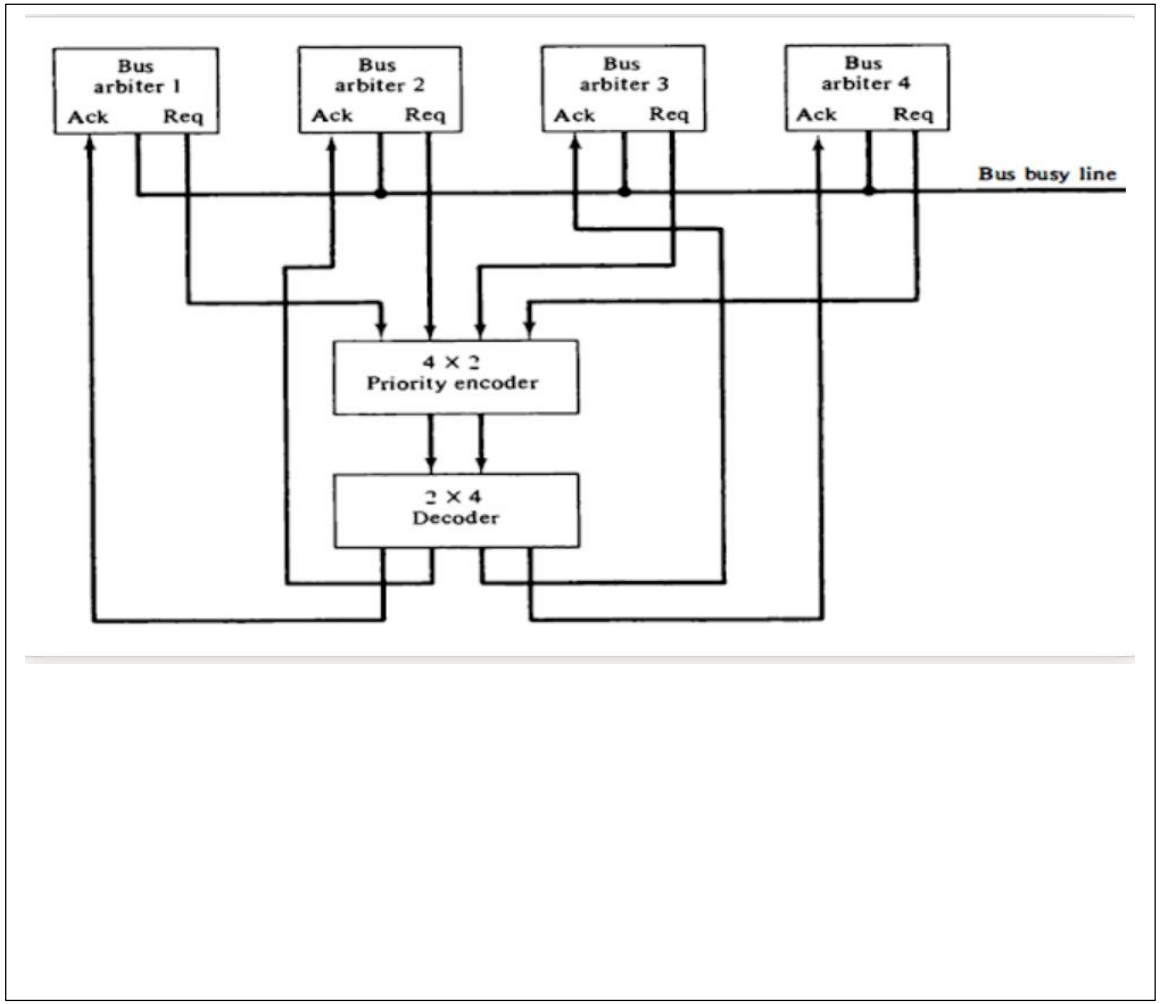


The PI of the highest-priority unit is maintained at a logic value 1. The highest-priority unit in the system will always receive access to the system bus when it requests it. The processor whose arbiter has a PI = 1 and PO = 0. That processor accesses the system bus.

Parallel Arbitration

In this technique uses an external priority encoder and decoder as shown in figure below. Each bus arbiter in the parallel scheme has a bus request output line and a bus acknowledge input line. When processor wants to access system bus at that time arbiter of that processor enables request line. The processor takes control of the bus if it acknowledges input line is enabled.

Figure shows the request lines from four arbiters going into a 4 x 2 priority encoder. The output of the encoder generates a 2-bit code, which represents the highest-priority unit among those requesting the bus. The 2-bit code from the encoder output drives a 2×4 decoder which enables the proper acknowledge line to grant bus access to the highest-priority unit. It works on priority encoder truth table.



Dynamic Arbitration Techniques

Serial and Parallel bus arbitration are static since the priorities assigned are fixed. In dynamic arbitration, priorities of the system change while the system is in operation. The various algorithms used are:-

Time Slice

It allocates a fixed-length time slice of bus time offered sequentially to each process in a round-robin fashion. The service provided by each system component and the system is independent of its location near the bus.

Polling

In polling, the controller generates the addresses for the devices. The number of address lines needed depends upon the number of connected devices to the system. The controller generates a sequence of device addresses in response to the bus request. When the requesting device recognizes its address, it activates the busy bus line and uses the bus. After several bus cycles, the polling process continues by choosing a different processor. The polling sequence usually is programmable, and as a result, the selection priority can be altered under program control.

Least Recently Used(LRU)

This algorithm provides the highest priority for a device, requesting that it has not used the bus for too long. The priorities are adjusted after several bus cycles according to the LRU algorithm.

FIFO(First IN First Out)

Requests are served in the order received in the FIFO scheme. The bus controller establishes a queue as per the request of the incoming bus to use this algorithm.

Each processor has to wait for its bus usage time as per first in and first-out (FIFO).

Rotating Daisy chain

It is a dynamic extension of the daisy chain algorithm. There is no central bus controller in this scheme, and the priority line is connected from the priority out of the last device back to the priority-in of the first device in a closed-loop.

Each arbiter priority for a given bus cycle is determined by its position along the bus priority line from the arbiter whose processor is currently controlling the bus.

Interprocessor Communication and Synchronization

The various processors in a multiprocessor system must be provided with a facility for communicating with each other.

• In a shared memory multi-processor system, the most common procedure is to set aside a portion of memory that is accessible to all processors.

•The primary use of common memory is to act as a message center similar to a mailbox, where each processor can leave messages for other processors and pick up messages intended for it.

• The sending processor structures a request, a message or a procedure and places it in the memory mail box, whether it has meaningful information, and which processor it is intended. The receiving processor can check the mailbox periodically if there are valid messages for it.

•In addition to shared memory, a multiprocessor system may have other shared resources. To prevent conflict use of shared resources by several processors there must be a provision for assigning resources to these processors. This task is given to the operating system.

Inter processor Communication and Synchronization

Interprocess Communication (IPC) refers to the exchange of data between two or more cooperating processes executing on different processors in a multiprocessor system.

The instruction set of a multiprocessor contains basic instructions that are used to Implement communication and synchronization between cooperating processes.

- Communication refers to the exchange of data between different processes.
- Synchronization refers to the special case where the data used to communicate between processors is control information.

Synchronization is needed to enforce the correct sequence of processes and to ensure mutually exclusive access to shared writable data.

Multiprocessor systems usually include various mechanisms to deal with the synchronization of resources.

Low-level primitives are implemented directly by the hardware.

These primitives are the basic mechanisms that enforce mutual exclusion for more Complex mechanisms implemented in software.

Synchronization is needed to enforce the correct sequence of processes and to ensure mutually exclusive access to shared writable data. A number of hardware mechanisms for mutual exclusion have been developed. One of the most popular methods is binary semaphore.

Critical section

Mutual exclusion must be provided in a multiprocessor system to enable one processor to exclude or lock out access to a shared resource by other processors when it is in a critical section.

- A critical section is a program sequence that, once begun, must complete execution before another processor accesses the same shared resource.
- A binary variable called a semaphore is often used to indicate whether or not a processor is executing a critical section

Mutual Exclusion with Semaphore

- A properly functioning multiprocessor system must provide a mechanism that will guarantee orderly access to shared memory and other shared resources.
- Mutual exclusion: This is necessary to protect data from being changed simultaneously by two or more processors.
- Critical section: is a program sequence that must complete execution before another processor accesses the same shared resource.
- A binary variable called a semaphore is often used to indicate whether or not a processor is executing a critical section.
- Testing and setting the semaphore is itself a critical operation and must be performed as a single indivisible operation.
- A semaphore can be initialized by means of a test and set instruction in conjunction with a hardware lock mechanism.

Working of Semaphore

A semaphore is a software controlled flag that is stored in a memory location that all processors can access.

When the semaphore is equal to 1, it means that a processor is executing a critical program, so that the shared memory is not available to other processors.
When the semaphore is equal to 0, the shared memory is available to any requesting processor.

Processors that share the same memory segment agree by convention not to use the memory segment unless the semaphore is equal to 0, indicating that memory is available. They also agree to set the semaphore to 1 when they are executing a critical section and to clear it to 0 when they are finished

ThankYou